

SOFTWARE REQUIREMENTS SPECIFICATION

NeoBank Platform — Sprint 1 — Foundation & Core Banking

Project	NeoBank Platform
Document	Software Requirements Specification (SRS)
Sprint	Sprint 1 — Foundation & Core Banking
Duration	10 Working Days (within 60-Day Lifecycle)
Program	PMIS Internship — Infosys Limited
Location	Bhubaneswar Development Centre — Lab 4 & Lab 5
Version	1.0 — Draft, Internal Review
Date	9 April 2026
Status	Active — Sprint In Progress
Next Milestone	Milestone 3 — Budgeting & Expense Management

© Infosys Limited · Confidential · Not for External Distribution

1. INTRODUCTION

The NeoBank Platform is an enterprise-grade digital banking solution developed under the PMIS Internship Program at the Infosys Bhubaneswar Development Centre, Lab 4 and Lab 5. This document is the authoritative Software Requirements Specification (SRS) for Sprint 1, covering the first 10 working days of a 60-day development lifecycle.

NeoBank resolves the fragmented nature of traditional banking by delivering a unified, centralised view of accounts, transactions, and financial activity. The platform adopts a modern decoupled full-stack architecture: Angular on the frontend, Spring Boot on the backend, MySQL as the persistence layer, and JWT-based stateless authentication for security.

1.1 Purpose

This SRS defines the complete set of functional requirements, system behaviour, business rules, integration strategy, quality assurance plan, and sprint-level execution roadmap for NeoBank Sprint 1. It serves as the binding technical reference for all interns, mentors, and stakeholders in Lab 4 and Lab 5.

1.2 Scope of Sprint 1

Sprint 1 is strictly scoped to the foundational architecture. Deliverables are limited to:

- ▶ FR-1: Authentication and User Management — registration, login, RBAC, profile management
- ▶ FR-2: Account and Transaction Management — account creation, transaction processing, real-time balance tracking
- ▶ Core infrastructure: environment setup, MySQL schema provisioning, Git repository initialisation
- ▶ Frontend-backend integration: Angular wired to Spring Boot REST APIs via JWT interceptors
- ▶ Unit and end-to-end testing of all sprint deliverables
- ▶ Swagger/OpenAPI documentation for all completed endpoints

1.3 Project Context

Internship Program	PMIS (Project Management Information System)
Organisation	Infosys Limited
Development Centre	Bhubaneswar DC
Labs	Lab 4 & Lab 5
Project	NeoBank Platform
Sprint Duration	10 Working Days
Total Cycle	60 Working Days
Next Sprint	Milestone 3 — Budgeting & Expense Management

1.4 Definitions & Abbreviations

TERM	DEFINITION
SRS	Software Requirements Specification — this document
FR	Functional Requirement — a categorised set of deliverable features
JWT	JSON Web Token — signed stateless token used for API authentication
RBAC	Role-Based Access Control — access restrictions based on assigned roles
DDL	Data Definition Language — SQL scripts to create database schema
API	Application Programming Interface — RESTful HTTP endpoints
ORM	Object-Relational Mapping — Spring Data JPA maps Java entities to MySQL
CORS	Cross-Origin Resource Sharing — browser security for cross-domain API calls
PMIS	Project Management Information System — the Infosys internship program
DC	Development Centre — Infosys Bhubaneswar facility

2. FUNCTIONAL REQUIREMENTS OVERVIEW

Sprint 1 delivers two core Functional Requirement modules. These form the architectural backbone of NeoBank and are prerequisites for all subsequent development milestones. Both modules must be fully functional and tested before sprint sign-off.

2.1 FR-1: Authentication & User Management

This module manages the complete user lifecycle. It is implemented first as it provides the security foundation upon which FR-2 and all future modules depend.

2.1.1 User Registration

- ▶ New users self-register with full name, email, and password
- ▶ Server-side email uniqueness validation — duplicates rejected with HTTP 409 Conflict
- ▶ Password complexity enforced before account creation
- ▶ Passwords stored exclusively as BCrypt hashes — plain-text never persisted
- ▶ Successful registration returns HTTP 201 Created with user profile (password excluded)

2.1.2 Secure Login & JWT Issuance

- ▶ Registered users authenticate via POST /api/auth/login with email and password
- ▶ On success, a signed JWT is generated with userId, email, role, and expiry claims
- ▶ JWT returned in response body; client stores it for subsequent API requests
- ▶ Failed login returns HTTP 401; inactive user login returns HTTP 403

2.1.3 Role-Based Access Control

- ▶ Two roles supported: ADMIN and CUSTOMER
- ▶ Admin routes protected with @PreAuthorize("hasRole('ADMIN')")
- ▶ CUSTOMER role accessing admin routes receives HTTP 403 Forbidden
- ▶ Role embedded in JWT and extracted per request by the JwtAuthFilter

2.1.4 Profile Management

- ▶ Authenticated users retrieve their profile via GET /api/users/me
- ▶ Editable fields updated via PUT /api/users/me
- ▶ Angular profile UI renders admin controls conditionally based on role

2.2 FR-2: Account & Transaction Management

FR-2 is the core banking engine. It enables account creation, transaction processing, and real-time balance tracking — all secured by the JWT layer from FR-1.

2.2.1 Account Creation & Viewing

- ▶ Authenticated users create accounts via POST /api/accounts

- ▶ Ownership is determined exclusively from the JWT payload — never from the request body
- ▶ System auto-generates a unique account number per account
- ▶ GET /api/accounts returns only accounts owned by the requesting user
- ▶ Cross-user account access rejected with HTTP 403 Forbidden

2.2.2 Transaction History Logging

- ▶ All debit and credit transactions are permanently recorded in the transactions table
- ▶ Records are immutable — no update or delete operations permitted
- ▶ GET /api/accounts/{id}/transactions returns paginated history, sorted date descending
- ▶ Each record includes: type, amount, description, date, and post-transaction balance snapshot

2.2.3 Balance Tracking & Overdraft Prevention

- ▶ Balances updated atomically with each transaction using @Transactional
- ▶ Pre-debit check: if requested amount > current balance → reject with HTTP 422
- ▶ Angular dashboard reflects real-time balance fetched directly from MySQL

3. BUSINESS & VALIDATION RULES

The following rules are hard constraints enforced at the backend API layer, independent of any frontend validation. Any API response that violates these rules is classified as a defect.

FR-1 — AUTHENTICATION RULES

REF	CONSTRAINT
BR-01	Email addresses must be absolutely unique. Duplicate registration attempts return HTTP 409 Conflict with a descriptive error body.
BR-02	Passwords must meet complexity standards: minimum 8 characters, at least one uppercase, one lowercase, one digit, and one special character. Non-compliant passwords return HTTP 400.
BR-03	A valid, non-expired JWT Bearer token must be present in the Authorization header for all secured endpoints. Missing or invalid tokens return HTTP 401 Unauthorized.
BR-04	JWT tokens must carry userId, email, and role claims. Expiry is mandatory (recommended: 24 hours). Expired tokens must be rejected with HTTP 401.
BR-05	Users flagged is_active = false must be blocked from login regardless of credential validity. Return HTTP 403 Forbidden with an inactivity message.
BR-06	Passwords must be stored exclusively as BCrypt hashes. Plain-text passwords must never be stored, logged, or returned in any API response.

FR-2 — ACCOUNT & TRANSACTION RULES

REF	CONSTRAINT
BR-01	Every account must be bound to the authenticated user. The userId must be extracted from the Spring SecurityContext — never from the HTTP request body.
BR-02	Debit transaction amount cannot exceed current available balance. Such requests are rejected with HTTP 422 Unprocessable Entity before any database write occurs.
BR-03	All transaction amounts must be strictly greater than zero. Zero or negative amounts return HTTP 400 Bad Request.
BR-04	Operations referencing invalid, non-existent, or unauthorised account IDs must be explicitly rejected: HTTP 404 for not found, HTTP 403 for unauthorised access.
BR-05	Transaction records are immutable once written. No PUT or DELETE endpoints shall exist for transaction resources.
BR-06	Account listing and retrieval APIs must return only resources owned by the authenticated user. Cross-user data access is strictly prohibited.

4. INFRASTRUCTURE & ARCHITECTURE

NeoBank uses a modern, loosely-coupled, three-tier full-stack architecture. Each layer is independently deployable and communicates via RESTful HTTP APIs.

4.1 Technology Stack

LAYER	TECHNOLOGY	DETAILS
Frontend	Angular	TypeScript SPA; Angular CLI; Reactive Forms; HTTP Interceptors; Route Guards
Backend	Spring Boot	Java REST API; Spring Security; Spring Data JPA; Hibernate ORM; Maven
Database	MySQL 8.0+	Relational RDBMS; InnoDB engine; DDL scripts; SQL seed data
Security	JWT + BCrypt	Stateless token auth; Bearer scheme; BCrypt password hashing
Version Control	Git	Separate repos for frontend & backend; feature-branch workflow
API Docs	Swagger/OpenAPI	springdoc-openapi; interactive UI at /swagger-ui.html
IDE	IntelliJ + VSCode	IntelliJ for Spring Boot; VS Code + Angular Language Service

4.2 Database Schema

Three core tables provisioned on Day 1 via DDL scripts. Foreign key constraints enforced with InnoDB engine.

4.2.1 users

- id — BIGINT, PK, AUTO_INCREMENT
- email — VARCHAR(255), UNIQUE, NOT NULL
- password_hash — VARCHAR(255), NOT NULL
- full_name — VARCHAR(255), NOT NULL
- role — ENUM('ADMIN', 'CUSTOMER'), DEFAULT 'CUSTOMER'
- is_active — BOOLEAN, DEFAULT TRUE
- created_at — TIMESTAMP, DEFAULT CURRENT_TIMESTAMP

4.2.2 accounts

- id — BIGINT, PK, AUTO_INCREMENT
- user_id — BIGINT, FK → users(id), ON DELETE CASCADE
- account_number — VARCHAR(20), UNIQUE, NOT NULL
- balance — DECIMAL(15,2), DEFAULT 0.00
- account_type — ENUM('SAVINGS', 'CURRENT'), NOT NULL
- created_at — TIMESTAMP

4.2.3 transactions

- id — BIGINT, PK, AUTO_INCREMENT
- account_id — BIGINT, FK → accounts(id), ON DELETE RESTRICT
- type — ENUM('DEBIT', 'CREDIT'), NOT NULL
- amount — DECIMAL(15,2), NOT NULL, CHECK (amount > 0)
- description — VARCHAR(500)
- transaction_date — TIMESTAMP, DEFAULT CURRENT_TIMESTAMP
- balance_after — DECIMAL(15,2), NOT NULL

5. INSTALLATION & SETUP — DAY 1

Day 1 is dedicated entirely to environment provisioning. Both sessions must be completed before Day 2 development begins. A working environment is a hard prerequisite for all subsequent sprint activity.

5.1 Session 1: Development Environment

Software to Install

- ▶ Node.js LTS and npm — required for Angular CLI and build tooling
- ▶ Angular CLI: `npm install -g @angular/cli` — verify with `ng version`
- ▶ JDK 17 or above — verify with `java -version`
- ▶ Spring Boot project from start.spring.io — dependencies: Spring Web, Spring Security, Spring Data JPA, MySQL Driver, Lombok
- ▶ IntelliJ IDEA (backend) and Visual Studio Code with Angular Language Service (frontend)
- ▶ Git — configure `user.name` and `user.email` globally

Repository Initialisation

- ▶ Create: `neobank-backend` (Spring Boot) and `neobank-frontend` (Angular) Git repositories
- ▶ Add `.gitignore`: Java template for backend, Node template for frontend
- ▶ Branch strategy: `main` → `develop` → `feature/*` — all work on feature branches
- ▶ Share repository access with all team members and the PMIS mentor

5.2 Session 2: Database Provisioning

MySQL Configuration

- ▶ Install MySQL Community Server 8.0+
- ▶ Create schema: `CREATE DATABASE neobank_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;`
- ▶ Create user: `CREATE USER 'neobank_user'@'localhost' IDENTIFIED BY '<secure>';`
- ▶ Grant access: `GRANT ALL PRIVILEGES ON neobank_db.* TO 'neobank_user'@'localhost';`
- ▶ Configure `application.properties` with JDBC URL, credentials, and `hibernate.ddl-auto=validate`

DDL Execution Order

- ▶ `01_create_users_table.sql`
- ▶ `02_create_accounts_table.sql`
- ▶ `03_create_transactions_table.sql`
- ▶ Verify: `SHOW TABLES;` and `DESCRIBE <table_name>;` for each table

6. SPRINT 1 EXECUTION PLAN

The 10-day sprint enables parallel backend and frontend development. Integration sessions on Days 5 and 10 bring both layers together. Each session is approximately 3–4 hours of focused work.

6.1 Sprint Schedule

DAY	PHASE	SESSION 1	SESSION 2
1	Env Setup	Install Angular CLI, JDK, Spring Boot; init Git repos	Install MySQL; execute DDL scripts for all 3 tables
2	Onboarding	POST /api/auth/register — email & password validation	Angular registration form wired to backend
3	Security	POST /api/auth/login — JWT generation, Spring Security config	Angular login UI; route guards; JWT interceptor
4	Access Control	RBAC via @PreAuthorize on admin routes	Angular profile management UI with role-aware rendering
5	Integration+QA	Wire Angular auth to Spring Boot; validate JWT in HTTP headers	Unit tests: JWT, BCrypt, endpoint security; e2e login flow
6	Accounts	POST & GET /api/accounts — ownership from JWT SecurityContext	Cross-user access rejection; ownership enforcement tests
7	Transactions	GET & POST /api/accounts/{id}/transactions — overdraft logic	Atomic balance update with @Transactional; edge case tests
8	Dashboard UI	Angular account creation & listing screens	Real-time balance display; integration with account APIs
9	Transaction UI	Angular transaction history data grid	DEBIT/CREDIT colour coding; client-side filter & sort
10	Integration+QA	Connect Angular dashboard to all banking APIs; e2e data flow validation	Unit tests for banking logic; e2e account & transaction tests

6.2 Day 2: User Onboarding

Session 1 — Backend: POST /api/auth/register

- ▶ Create User JPA entity with @Entity, @Table, @Id, @Column(unique=true)
- ▶ UserRepository extends JpaRepository<User, Long>
- ▶ UserService.register(): uniqueness check → BCrypt hash → save → return DTO
- ▶ AuthController @PostMapping("/api/auth/register") → 201 Created or 409/400

Session 2 — Frontend: Registration

- ▶ ng generate component auth/register
- ▶ Reactive FormGroup: fullName, email, password, confirmPassword

- ▶ Validators: required, email format, minLength(8), custom passwordMatch
- ▶ AuthService.register() via HttpClient; display 201 success or 400 field errors

6.3 Day 3: Security Implementation

Session 1 — Backend: Login & JWT

- ▶ UserDetailsServiceImpl loads user by email
- ▶ JwtUtil: generateToken(), validateToken(), extractClaims()
- ▶ POST /api/auth/login: authenticate → generate JWT → return in response body
- ▶ JwtAuthFilter: validate Bearer token per request, populate SecurityContext
- ▶ SecurityFilterChain: permit /api/auth/**, authenticate all other routes

Session 2 — Frontend: Login UI & Guards

- ▶ ng generate component auth/login; call AuthService.login() on submit
- ▶ AuthService stores JWT in sessionStorage; clears on logout
- ▶ AuthGuard implements CanActivate: redirect to /login if no valid token
- ▶ JwtInterceptor implements HttpInterceptor: attach Authorization: Bearer <token> automatically

6.4 Day 4: Access Control

Session 1 — RBAC

- ▶ @EnableMethodSecurity in SecurityConfig
- ▶ @PreAuthorize("hasRole('ADMIN')") on admin-only endpoints
- ▶ Role extracted from JWT claims in JwtAuthFilter → GrantedAuthority in SecurityContext
- ▶ HTTP 403 Forbidden with descriptive message for unauthorised role access

Session 2 — Profile Management

- ▶ ng generate component user/profile
- ▶ GET /api/users/me to fetch full profile; PUT /api/users/me for updates
- ▶ *ngIf conditionally renders admin controls based on decoded JWT role claim

6.5 Day 6: Account Module

- ▶ Create Account entity, AccountRepository, AccountService
- ▶ POST /api/accounts: userId from SecurityContextHolder (never request body) → 201 Created
- ▶ GET /api/accounts: WHERE user_id = authenticated user only
- ▶ GET /api/accounts/{id}: ownership check → 403 if mismatch, 404 if not found
- ▶ Auto-generate unique account numbers (UUID-based)

6.6 Day 7: Transaction Engine

- ▶ Create Transaction entity, TransactionRepository, TransactionService

- ▶ GET /api/accounts/{id}/transactions: ownership validated, paginated, date desc
- ▶ POST /api/accounts/{id}/transactions: validate ownership → validate amount > 0 → overdraft check → @Transactional update balance + insert record
- ▶ balance_after stored as immutable snapshot per transaction for auditability

6.7 Days 8 & 9: Frontend UI

Day 8 — Dashboard

- ▶ Account list: account number, type, balance for each owned account
- ▶ Account creation form: type dropdown (SAVINGS / CURRENT); calls POST /api/accounts
- ▶ Real-time balance: refresh via EventEmitter or observable after each transaction

Day 9 — Transaction UI

- ▶ Transaction grid columns: Date, Type, Amount, Description, Balance After
- ▶ Colour coding: CREDIT → green (#2E7D32), DEBIT → red (#C62828) via Angular class binding
- ▶ Client-side filter (All / DEBIT / CREDIT) and sort by date (asc/desc)
- ▶ Debit/credit form: amount input (min 0.01), description, type selection

7. API REFERENCE

All secured endpoints require a valid JWT Bearer token in the Authorization header. Unsecured endpoints are accessible without authentication.

FR-1 — AUTHENTICATION ENDPOINTS

ENDPOINT	METHOD	DAY	AUTH	DESCRIPTION
POST /api/auth/register	POST	Day 2	Open	Register new user with email uniqueness & password complexity checks. Returns 201.
POST /api/auth/login	POST	Day 3	Open	Authenticate credentials, return signed JWT. Returns 401 or 403 on failure.
GET /api/users/me	GET	Day 4	JWT	Retrieve authenticated user profile.
PUT /api/users/me	PUT	Day 4	JWT	Update profile fields for authenticated user.
GET /api/admin/users	GET	Day 4	ADMIN	List all system users. Admin role only.

FR-2 — ACCOUNT & TRANSACTION ENDPOINTS

ENDPOINT	METHOD	DAY	AUTH	DESCRIPTION
POST /api/accounts	POST	Day 6	JWT	Create bank account; ownership set from JWT. Returns 201.
GET /api/accounts	GET	Day 6	JWT	List all accounts owned by authenticated user.
GET /api/accounts/{id}	GET	Day 6	JWT	Get account details. 403 if not owner; 404 if not found.
GET /api/accounts/{id}/transactions	GET	Day 7	JWT	Paginated transaction history sorted by date descending.
POST /api/accounts/{id}/transactions	POST	Day 7	JWT	Create debit/credit. Enforces overdraft check and amount > 0.

8. INTEGRATION STRATEGY

Integration sessions on Days 5 and 10 bridge Angular and Spring Boot. These sessions validate end-to-end data flow from the user interface through the API layer to MySQL and back.

8.1 Day 5 — Authentication Integration

- ▶ Wire Angular registration and login forms to Spring Boot endpoints via AuthService
- ▶ Confirm JWT is returned in login response, stored in sessionStorage, and cleared on logout
- ▶ Verify JwtInterceptor attaches Authorization: Bearer <token> on all outgoing requests
- ▶ Test unauthenticated requests to secured endpoints return HTTP 401 in browser network tab
- ▶ Validate complete journey: Register → Login → JWT stored → Guarded route accessed → Logout → Redirect
- ▶ Confirm CORS configuration in SecurityConfig permits Angular dev server origin

8.2 Day 10 — Banking Integration

- ▶ Connect Angular account dashboard to POST /api/accounts and GET /api/accounts
- ▶ Wire transaction UI to GET and POST /api/accounts/{id}/transactions
- ▶ Verify balance in Angular matches MySQL after each transaction (cross-reference in DevTools + DB)
- ▶ Validate real-time balance refresh without full page reload after transaction creation
- ▶ Test error scenarios in UI: overdraft rejection, invalid account, expired JWT

9. DATA SEEDING

SQL seeding scripts populate the database after schema creation, enabling integration and QA sessions to proceed with consistent, representative test data.

ENTITY	COUNT	SEED DETAILS
Admin User	1	admin@neobank.in / Admin@123 — ADMIN role, is_active = true
Customer Users	3	customer1–3@neobank.in — CUSTOMER role, is_active = true
Inactive User	1	inactive@neobank.in — CUSTOMER role, is_active = false (tests login block)
Bank Accounts	6+	Mix of SAVINGS and CURRENT distributed across the 3 customer users
Sample Transactions	12+	Mix of DEBIT and CREDIT with varying amounts across accounts
Categories	N/A	Groceries, Salary, Utilities, Entertainment, Transfer — dropdown population

10. QUALITY ASSURANCE & TESTING

QA is integrated into Days 5 and 10. Testing is split between the authentication module (Day 5) and the banking module (Day 10). All tests must pass before sprint sign-off.

DAY 5 — FR-1 AUTHENTICATION QA

Unit Tests

- ▶ JWT generation: assert token contains correct claims; assert expiry is within expected window
- ▶ JWT validation: assert expired tokens rejected; tampered tokens fail signature verification
- ▶ BCrypt: assert hash is non-reversible; assert BCrypt.checkpw(plain, hash) returns true
- ▶ Registration — duplicate email: assert HTTP 409; weak password: assert HTTP 400
- ▶ Login — invalid credentials: HTTP 401; inactive user: HTTP 403
- ▶ Secured endpoint without token: HTTP 401; CUSTOMER role on admin route: HTTP 403

End-to-End Tests

- ▶ Full Registration → Login → Protected Route navigation flow in browser
- ▶ Verify JWT in Authorization header via browser DevTools network inspector
- ▶ Verify Angular route guard blocks direct /dashboard URL navigation without login

DAY 10 — FR-2 BANKING QA

Unit Tests

- ▶ Overdraft prevention: debit amount > balance returns HTTP 422
- ▶ Zero/negative amount: amount ≤ 0 returns HTTP 400
- ▶ Ownership check: different user JWT accessing an account returns HTTP 403
- ▶ Invalid account ID: non-existent accountId returns HTTP 404
- ▶ Balance accuracy: balance = initial + credits – debits after multiple transactions
- ▶ Immutability: confirm no PUT or DELETE endpoint exists for transactions

End-to-End Tests

- ▶ Login → Create Account → 2 credits → 1 debit → view history → verify balance
- ▶ Overdraft attempt from Angular UI: verify error message displayed correctly
- ▶ Verify colour coding in transaction grid: green for CREDIT, red for DEBIT
- ▶ Cross-verify Angular balance against SELECT balance FROM accounts WHERE id = ? in MySQL

11. DOCUMENTATION & SPRINT WRAP-UP

Day 10 concludes with a documentation and sign-off session. All deliverables below must be committed to Git before the sprint is considered closed.

11.1 API Documentation

- ▶ Add springdoc-openapi-starter-webmvc-ui to pom.xml
- ▶ Annotate controllers and DTOs with @Operation, @ApiResponse, @Schema
- ▶ Swagger UI accessible at <http://localhost:8080/swagger-ui.html>
- ▶ Export OpenAPI JSON: <http://localhost:8080/v3/api-docs> → commit as docs/openapi-sprint1.json

11.2 Frontend Documentation

- ▶ Document all Angular components: purpose, inputs/outputs, services injected, routes
- ▶ Document environment variables in environment.ts and environment.prod.ts
- ▶ Document routing structure and all applied route guards
- ▶ Write README.md with complete local setup instructions

11.3 Sprint Review & Sign-Off

- ▶ Live demonstration of all Sprint 1 deliverables against acceptance criteria in this SRS
- ▶ Code review of both repositories with PMIS mentor
- ▶ QnA session for Lab 4 and Lab 5 intern teams
- ▶ Formal mentor sign-off upon acceptance of all deliverables
- ▶ Workspace preparation for Milestone 3: Budgeting & Expense Management

12. ACCEPTANCE CRITERIA

All criteria below constitute the Definition of Done for Sprint 1. Every item must be met before the sprint can be signed off.

FR-1 — AUTHENTICATION & USER MANAGEMENT

#	CRITERION	VERIFIED BY
AC-01	Registration API enforces email uniqueness and password complexity	Unit Test
AC-02	Login API returns a valid, correctly structured JWT on success	Unit Test + Postman
AC-03	JWT validated on all secured endpoints; missing token returns 401	Unit Test

#	CRITERION	VERIFIED BY
AC-04	RBAC enforced — CUSTOMER role blocked from admin routes with 403	Unit Test
AC-05	Inactive users blocked from login with 403	Unit Test
AC-06	Angular registration, login, and profile UIs functional and integrated	E2E Test
AC-07	Route guards prevent unauthenticated direct URL access to protected pages	E2E Test

FR-2 — ACCOUNT & TRANSACTION MANAGEMENT

#	CRITERION	VERIFIED BY
AC-08	Account creation ties ownership exclusively to JWT payload userId	Unit Test
AC-09	Account listing returns only the authenticated user's own accounts	Unit Test
AC-10	Debit exceeding balance rejected with HTTP 422	Unit Test
AC-11	Transaction history is accurate and returned date-descending	Unit Test + Postman
AC-12	Angular dashboard displays accounts, balances, and transaction history correctly	E2E Test
AC-13	DEBIT colour-coded red, CREDIT colour-coded green in transaction grid	E2E Test

GENERAL QUALITY

#	CRITERION	VERIFIED BY
AC-14	All unit tests pass with zero critical failures across FR-1 and FR-2	Test Suite
AC-15	Swagger UI accessible; all endpoints documented with request/response schemas	Manual Review
AC-16	Git repositories clean: feature branches merged, commit messages descriptive	Mentor Review
AC-17	No hardcoded credentials; code follows Infosys formatting standards	Code Review
AC-18	README.md files present in both repos with complete setup instructions	Mentor Review

Prepared under the PMIS Internship Program · Infosys Bhubaneswar DC · Lab 4 & Lab 5
© Infosys Limited · Confidential · Not for External Distribution